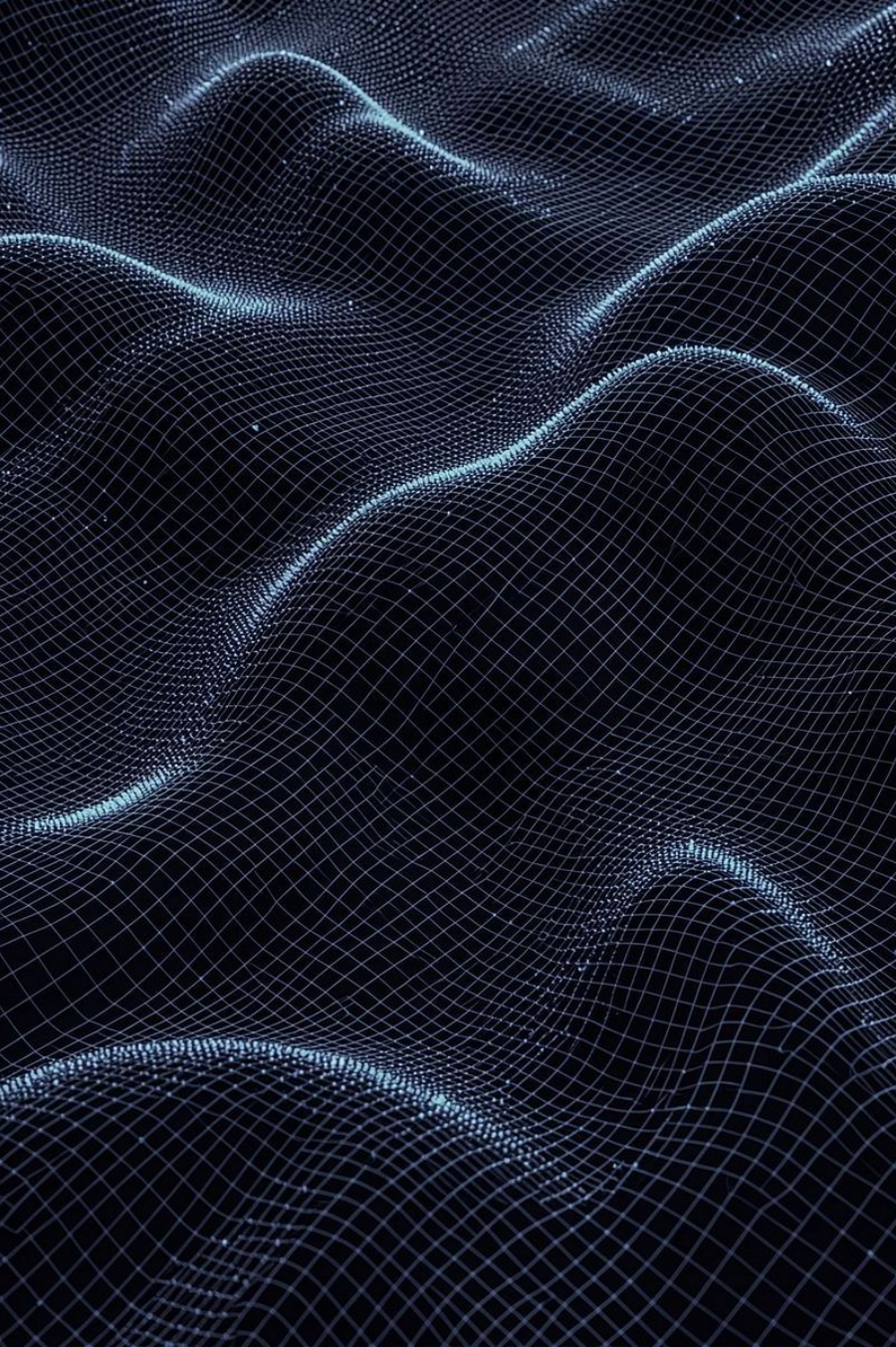




Evaluation and Context Optimization of Retrieval- Augmented Generation for Document Question Answering

Term Paper Presentation

Melina Giri | Roll No: 8125018 · Supervisor: **Asst. Prof. Balkrishna Subedi**

CDCSIT, Tribhuvan University

Presentation Overview

01

Introduction

Background, problem statement and objectives

03

Methodology

Dataset, workflow, retrieval and generation configurations

05

Results & Analysis

Retrieval performance, lexical F1 and latency

02

Literature Review

RAG, dense retrieval and evaluation concepts

04

Implementation

Tools, technologies and system implementation

06

Conclusion

Key findings, limitations and future recommendations

Background & Problem Statement

The Problem: Document Information Overload

- Research documents are increasing rapidly
- Finding relevant information manually is time-consuming
- Traditional keyword search may return many irrelevant documents
- Language models alone may lack access to specific external documents

1.2 Problem Statement

- Too few retrieved sections → Important information may be missed
- Too many retrieved sections → Redundant or irrelevant context may be introduced
- More context does not always mean better answers
- Context optimization may remove useful information

The Solution: Retrieval-Augmented Generation

- Retrieves relevant document sections
- Supplies retrieved information to the language model
- Generates answers based on external document context

📌 **Key Idea:** Better retrieval + better context selection → more effective document QA

1.3 Objectives

- Evaluate retrieval performance at different **k values**
- Compare **top-3, top-10 and optimized top-10** configurations
- Evaluate generated answers using **lexical F1**
- Analyze **generation latency and context size**

Dataset & System Architecture

Vectara Open RAG Benchmark

1,000

PDF Research Papers

3,045

Valid QA Pairs

18,840

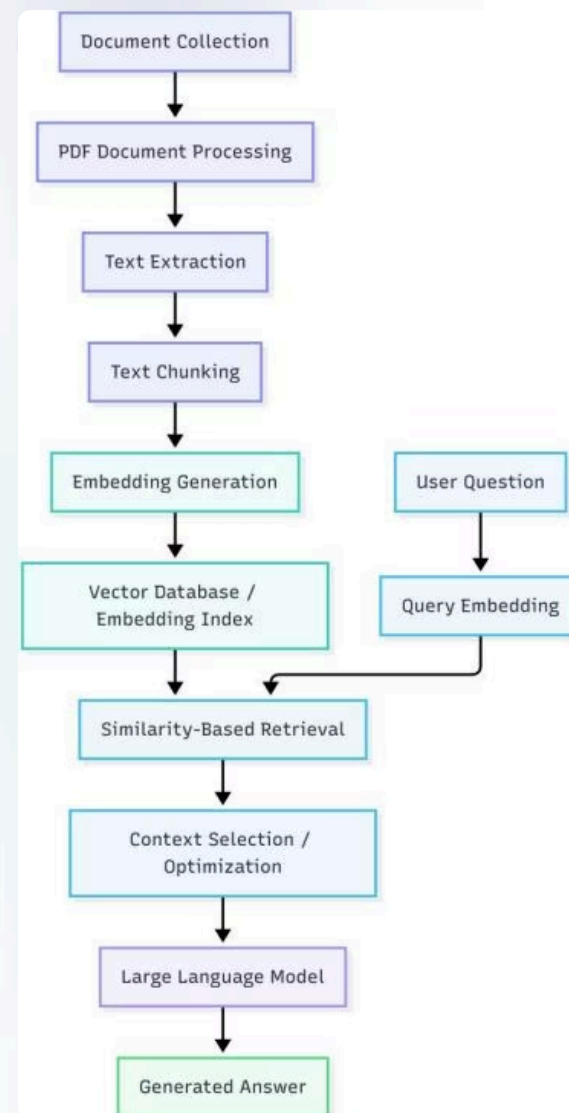
Document Sections

Question Types

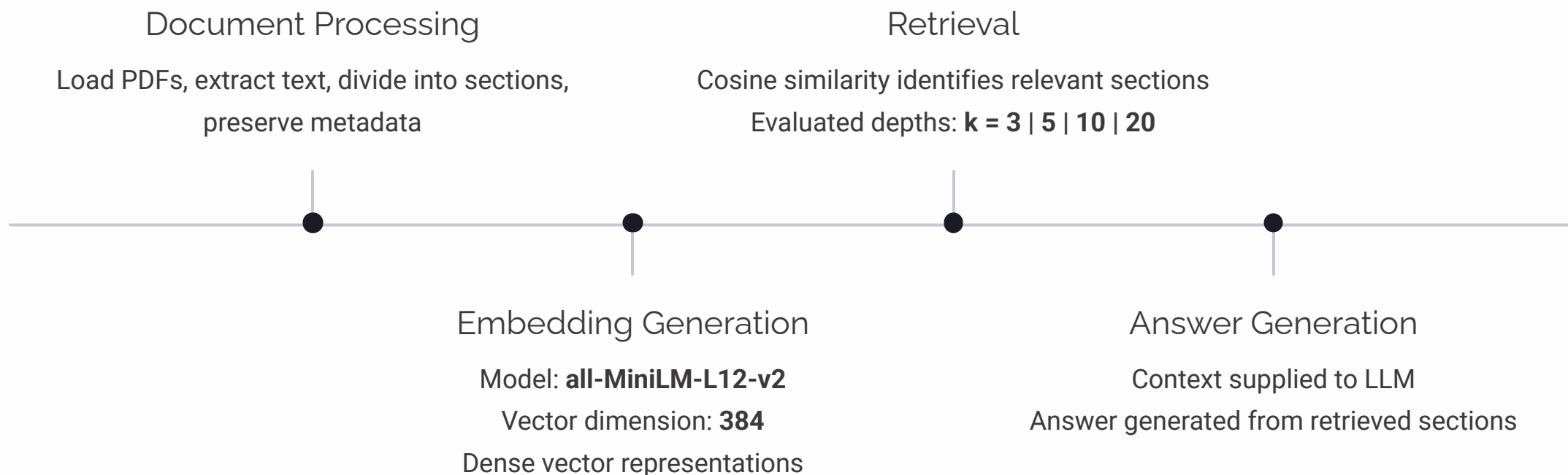
- **1,793 Abstractive**
- **1,252 Extractive**

Source Categories

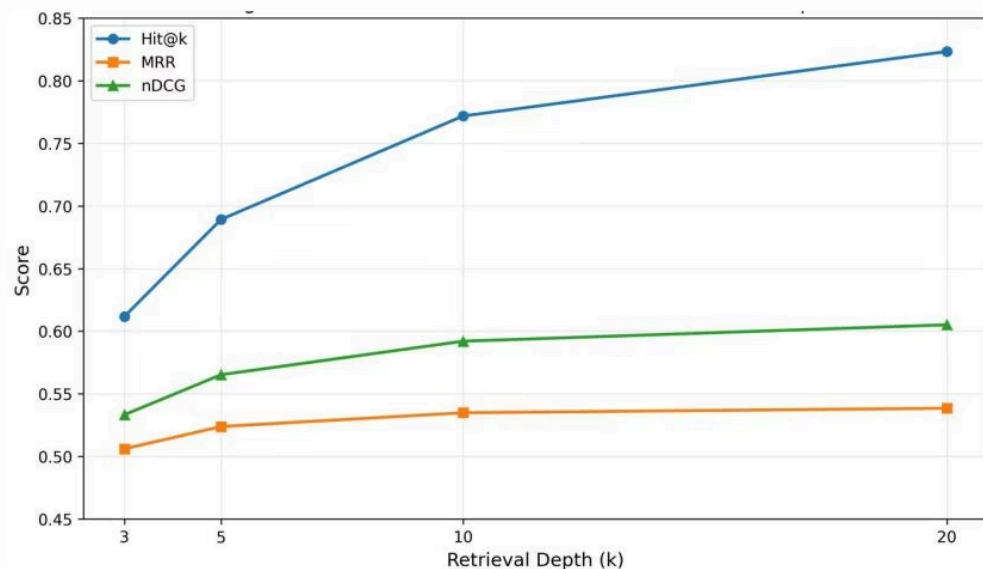
- Text · Text-Image · Text-Table · Text-Table-Image



Methodology: Processing, Embedding & Retrieval



Retrieval Results



Retrieval Performance by Depth

Depth	Hit@k	MRR	nDCG
k = 3	0.6118	0.5061	0.5333
k = 5	0.6893	0.5239	0.5653
k = 10	0.7718	0.5349	0.5920
k = 20	0.8233	0.5385	0.6051

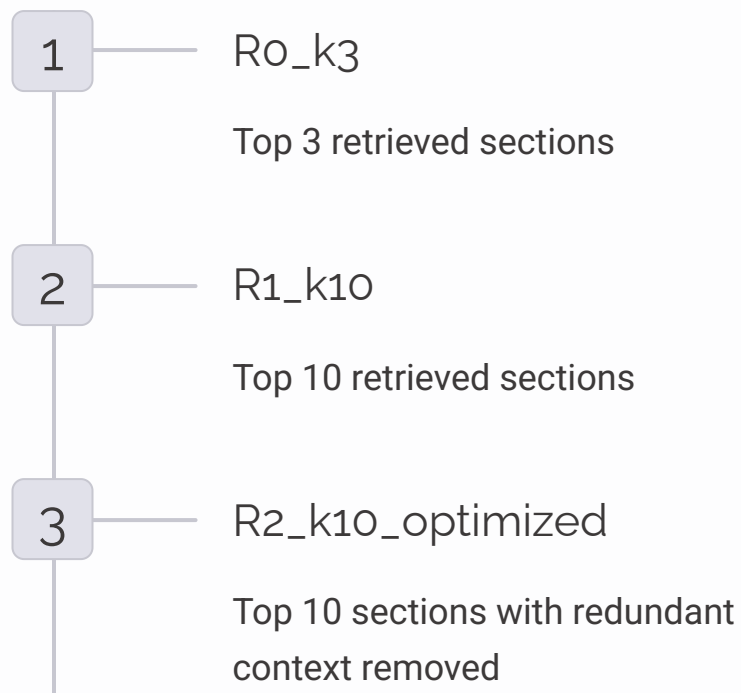
❏ **Hit@20 = 82.33% vs Hit@3 = 61.18%** – More retrieval depth → better coverage and ranking quality

Generation Experiment Setup

Experimental Setup

- **60 benchmark questions**
- **3 RAG configurations**

Configurations



Language Model & Evaluation

- Model: **Qwen2.5-3B**
- Locally executed using **Ollama**

Evaluation Metrics

- Lexical F1
- IDK percentage
- Generation latency
- Context size

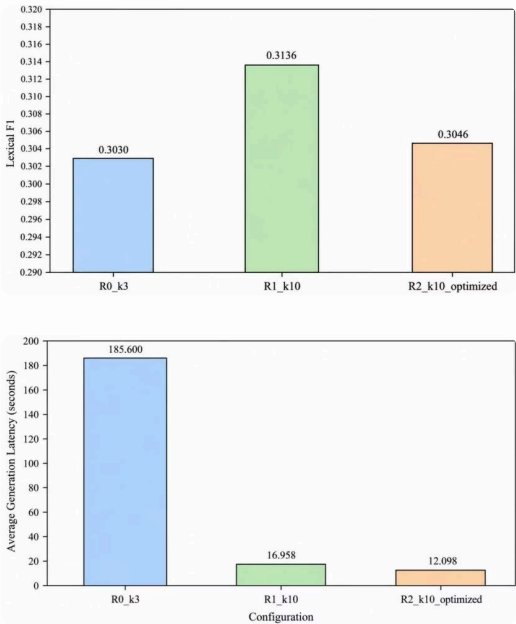
Retrieval Metrics

- **Hit@k** — Whether at least one relevant section appears in top-k results
- **MRR** — How highly the first relevant result is ranked
- **nDCG** — Overall ranking quality with greater importance given to highly ranked results

Generation Performance & F1 Analysis

Generation Performance Summary

Metric	R0_k3	R1_k10	R2_k10_opt.
Questions	60	60	60
Mean Lexical F1	0.3030	0.3136	0.3046
IDK %	23.33%	23.33%	25.00%
Mean Latency (s)	185.600	16.958	12.098
Context Sections	2.45	3.23	3.10



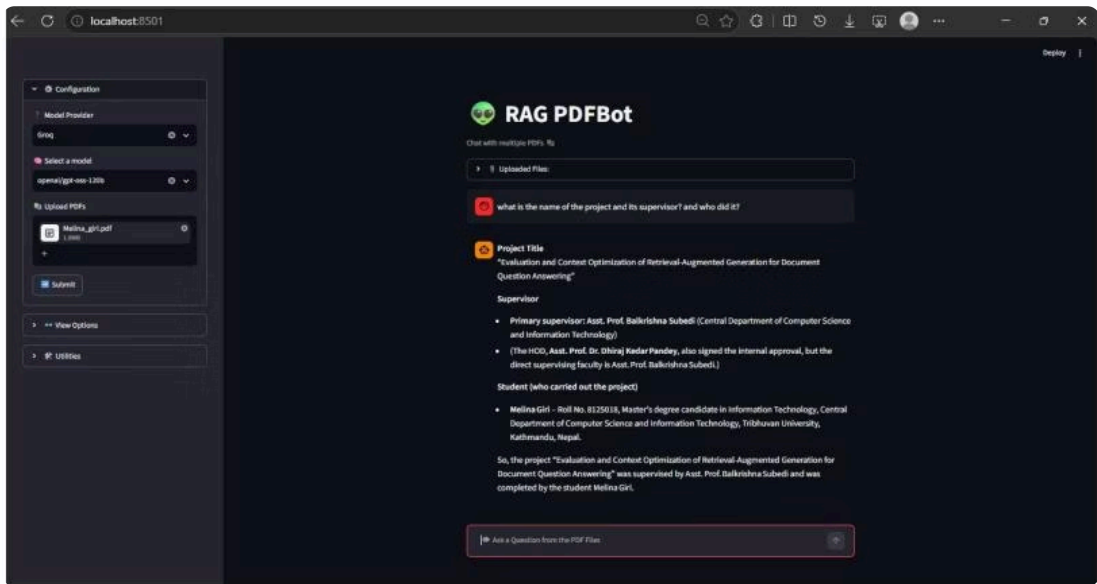
Paired Lexical-F1 Comparison

Comparison	Change
R0_k3 → R1_k10	+0.0106
R1_k10 → R2_k10_optimized	-0.0090
R0_k3 → R2_k10_optimized	+0.0016

📌 **Key Insight:** Context optimization improved efficiency but did not improve lexical F1. Lexical F1 measures word overlap – not definitive semantic correctness.

System Output & Implementation

RAG PDFBot — Actual System Output



The system retrieves relevant document context and uses it to generate a document-based answer.

Implementation Stack

Component	Technology
Programming	Python 3.x
Backend	FastAPI
Frontend	Streamlit
RAG Framework	LangChain
Embeddings	Sentence Transformers
Vector Database	ChromaDB
Embedding Model	all-MiniLM-L12-v2
Local LLM	Qwen2.5-3B
LLM Runtime	Ollama
Data Processing	NumPy, Pandas
Visualization	Matplotlib

Conclusion & Future Work

Major Findings

- **Retrieval depth matters** — Hit@20 increased to **82.33%**, compared with 61.18% at k=3
- **Top-10 retrieval achieved highest lexical F1** — R1_k10 = **0.3136**
- **Context optimization improved efficiency** — **12.098 s** average latency
- **Optimization did not improve lexical F1** — R2_k10_optimized = 0.3046
- **All 180 generations completed successfully** — 60 questions × 3 configurations

 **Key Insight:** Increasing retrieval depth improves information coverage, while context optimization can improve efficiency without necessarily improving answer quality.

Future Recommendations

- **Larger Generation Evaluation** — Evaluate more than 60 questions for stronger statistical evidence
- **Advanced Context Optimization** — Explore semantic similarity, relevance, diversity and learned ranking
- **Compare Embedding & Language Models** — Study effects on quality, hallucination, latency and resources
- **Semantic Answer Evaluation** — BERTScore · Embedding Similarity · LLM-as-a-Judge · Human Evaluation
- **Improve Retrieval & Ranking** — Reranking · Hybrid Retrieval · Query Expansion

References

- [1] Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020.
 - [2] Karpukhin et al., *Dense Passage Retrieval for Open-Domain Question Answering*, 2020.
 - [3] Reimers & Gurevych, *Sentence-BERT*, 2019.
 - [4] Vectara, *Open RAG Benchmark*.
 - [5] Qwen Team, *Qwen2.5: A Party of Foundation Models*, 2024.
 - [6] Sentence-Transformers, *all-MiniLM-L12-v2*, Hugging Face Model Card.
-

Thank You!

Questions & Discussion

Melina Giri | Roll No: 8125018

Supervisor: Asst. Prof. Balkrishna Subedi

Central Department of Computer Science and Information Technology, Tribhuvan University